Unité 2: Initiez-vous à Python - Activités de Blockly à Python

Site: NiryoAcademy
Cours: De Blockly à Python avec le robot Ned2
Livre: Unité 2: Initiez-vous à Python - Activités de Blockly à Python

Imprimé par: Theo Bernicot
Date: vendredi 16 janvier 2026, 15:31

Table des matières

Introduction à Python

Activité 1 – Comparer Blockly et Python

- Activité 1: Mouvement et outils
- Activité 2: Boucles et variables
- Activité 3: Boucles et convoyeur
- Activité 4: Conditions et fonctions

Activité 2 – Prédire le comportement du robot

- Challenge #1
- Challenge #2
- Challenge #3
- Challenge #4

Activités supplémentaires

Introduction à Python

Qu’est-ce que Python ?

Python est un **langage de programmation textuel**. Contrairement à Blockly, qui repose sur des blocs visuels à assembler, Python s’écrit en lignes de texte.

Ses principaux atouts :

- **Lisible** : proche du langage naturel, facile à comprendre.
- **Polyvalent** : utilisé pour la robotique, l’intelligence artificielle, le web, la data science.
- **Accessible** : recommandé comme premier langage pour débutants.

Avec le robot Ned2, Python permet :

- de programmer des mouvements complexes,
- d’automatiser des séquences,
- de connecter le robot à des capteurs ou à des librairies externes pour des projets avancés.

Pourquoi passer de Blockly à Python?

1. **Plus de liberté et de flexibilité** : créer des programmes plus complexes et manipuler des listes, boucles et fonctions avancées.
2. **Compréhension de la programmation réelle** : voir la syntaxe d’un vrai langage de programmation.
3. **Réutilisation et collaboration** : partager et modifier facilement des programmes Python.
4. **Transition vers le monde professionnel** : Python est largement utilisé dans la robotique et l’industrie.

💡 **À retenir :** Passer à Python, c’est passer de la logique visuelle à la logique textuelle, un pas vers la programmation professionnelle.

Quelles sont les principales différences ?

Blockly	**Python**
Langage **visuel** basé sur des blocs.	Langage **textuel** basé sur des lignes de code.
Pas d’erreurs de syntaxe (les blocs s'assemblent)	La syntaxe doit être respectée (indentation, ponctuation).
Fonctionnalités limitées à celles intégrées dans NiryoStudio.	Permet l’utilisation de **librairies externes** (pyniryo, OpenCV, TensorFlow…).
Idéal pour apprendre la logique.	Idéal pour aller plus loin et créer des projets complexes.

Qu'est ce qu'une librairie?

Une **librairie** est un ensemble de fonctions et outils prêts à l’emploi que l’on peut importer dans un programme Python.

> Exemples pour Ned2 :
>
> - `pyniryo` → contrôle du robot.
> - `opencv` → vision par ordinateur.
> - `tensorflow` → intelligence artificielle.

💡 **À retenir :** Les librairies permettent de se concentrer sur la logique de votre projet plutôt que de réécrire des fonctions déjà existantes.

Activité 1 – Comparer Blockly et Python

Dans ce bloc, nous allons proposer plusieurs activités qui auront cette structure:

1.Télécharger un programme Blockly et son équivalent en Python.

Vous pourrez les importer dans NiryoStudio et comparer les deux programmes, d'un onglet à l'autre.

2. Comparaison des programmes

Remplissez les tableaux d'équivalence pour vous familiariser avec quelques fonctions Python.

3. (Facultatif) Modification Python

Vous pourrez ensuite modifier les programmes directement en Python pour commencer à écrire en Python!

Activité 1: Mouvement et outils

Préparation:

Téléchargez les programmes suivants:

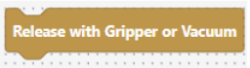
- Blockly 👉 Programme "Séquence" .json
- Python 👉 Programme "Séquence" .py

Contenu de l'activité:

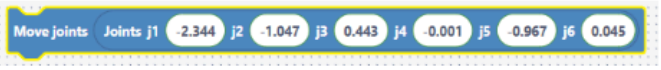
Vous êtes déjà familier du programme Blockly que nous avons créé ensemble dans la première partie de ce cours.

Vous pouvez maintenant vous familiariser avec le programme écrit en Python.

A vous de compléter le tableau d'équivalence suivant:

Blockly	**Python**
	
	
	

Explication:

Blockly	**Python**
	`robot.move(JointsPosition(-2.344, -1.047, 0.443, -0.001, -0.967, 0.045))`
	`robot.grasp_with_tool()`
	`robot.release_with_tool()`

💡 **Plus en détail**

```
robot.move(JointsPosition(-2.344, -1.047, 0.443, -0.001, -0.967, 0.045))
```

`move()`

- C'est une **méthode (fonction attachée à l'objet robot)** qui sert à **déplacer le robot**.
- Elle déplace **le robot dans une position spécifique**, selon les angles ou coordonnées passés en paramètre.

`JointsPosition(...)`

- C'est un objet qui représente **la position de tous les joints du robot**.
- Le Ned2 a 6 axes (joints), et chacun a un angle exprimé en **radians**.
- Dans cet exemple :

Joint Angle (radians)
1 -2.344

Joint	Angle (radians)
2	-1.047
3	0.443
4	-0.001
5	-0.967
6	0.045

- Ces valeurs indiquent **l'orientation exacte de chaque articulation** pour atteindre la position souhaitée.

Résumé en langage simple :

> « Le robot **se déplace vers la position définie par les angles de ses 6 joints**. »

Activité 2: Boucles et variables

Préparation:

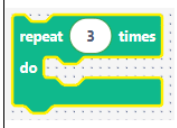
Téléchargez les programmes suivants:

- Blockly 👉 Programme "Boucle" .json
- Python 👉 Programme "Boucle" .py

Contenu de l'activité:

Comme précédemment, vous pouvez vous familiariser avec le programme Python.

1/ A vous de compléter le tableau d'équivalence suivant:

Blockly	Python
	
	
	

2/ **A vous de jouer!**

Dans le programme Python, apporter les modifications suivantes pour:

1. Placer 2 jetons bleus sur la plaque vision
2. Placer 3 jetons verts sur la plaque vision
3. BONUS: Placer 2 jetons bleu et 3 jetons vert sur la plaque vision

---

Explication

1/ Tableau d'équivalences

Blockly	Python
	`for count in range(3)`
	`position_prise_bleu = JointsPosition(-2.542, -0.948, 0.101, -0.002, -0.720, -1.651)`
	`(position_prise_bleu)`

💡 **Plus en détails**

```
for count in range(3)
```

`for`

- C'est une **boucle en Python** qui sert à répéter un bloc de code un certain nombre de fois.

- On l'utilise pour **parcourir une séquence** ou **répéter un code un nombre fixe de fois**.

`count`

- C'est la **variable de boucle**.
- À chaque itération (tour de boucle), `count` prend une **valeur différente** de la séquence générée par `range(3)`.
- Vous pouvez la nommer autrement (`i`, `index`, etc.), mais ici elle s'appelle `count`.

`range(3)`

- `range()` génère une **suite de nombres entiers**.
- `range(3)` crée la suite : `0, 1, 2` (attention : **le dernier nombre n'est pas inclus**).
- La boucle sera donc exécutée **3 fois**.

Résumé en langage simple :

> « Répétez le bloc de code **3 fois**, avec `count` prenant successivement les valeurs 0, 1 et 2. »

```
position_prise_bleu = JointsPosition(-2.542, -0.948, 0.101, -0.002, -0.720, -1.651)
```

`position_prise_bleu`

- C'est **le nom de la variable** que vous créez.
- Elle **contiendra la position exacte des joints du robot** pour attraper un objet bleu.
- Vous pouvez nommer la variable comme vous voulez, mais ici le nom est explicite.

`=`

- C'est l'opérateur **d'affectation**.
- Il sert à **attribuer la valeur de droite à la variable de gauche**.

`JointsPosition(...)`

- C'est un objet représentant **la position de tous les joints du robot**.

Résumé en langage simple :

> « La variable `position_prise_bleu` **contient les angles des 6 joints** nécessaires pour que le robot atteigne la position de prise de l'objet bleu. »

```
(position_approche_bleu)
```

C'est la variable que l'on a créé précédemment, stockée.

2/ Modifications du programme Python

Placer 2 jetons bleus sur la plaque vision:

La fonction à modifier est la suivante:

```
for count in range(3)
```

Au lieu de 3 répétitions, on modifie ici la valeur de la range, c'est-à-dire la plage de valeur. Le robot va compter dans cette plage de valeur (0, 1 et 2 dans l'exemple) chaque répétition.

Si on indique une plage de valeur "2", le robot répète deux fois et va donc prendre et déplacer 2 jetons bleus.

Placer 3 jetons verts sur la plaque vision

La fonction à modifier est la suivante:

```
robot.move(position_approche_bleu)
```

Au lieu de lui indiquer la variable (position_prise_bleu) on lui indique la variable (position_prise_vert).

On fait pareil pour la position d'approche.

```
robot.move(Position_approche_vert)
robot.move(Position_prise_vert)
```

BONUS: Placer 2 jetons bleus et 3 jetons verts sur la plaque vision

Il s'agit ici de reprendre les deux modifications qu'on a faite précédemment en faisant deux boucles:

- Boucle 1: répéter 2x l'action de prendre et déplacer le jeton bleu
- Boucle 2: répéter 3x l'action de prendre et déplacer le jeton vert.

```
# !/usr/bin/env python3
from pyniryo import NiryoRobot, JointsPosition

robot = NiryoRobot('127.0.0.1')

robot.calibrate_auto()

position_prise_bleu = JointsPosition(-2.542, -0.948, 0.101, -0.002, -0.720, -1.651)
Position_prise_vert = JointsPosition(-2.357, -1.199, 0.593, -0.001, -0.963, 0.70)
Position_approche_vert = JointsPosition(-2.343, -1.047, 0.442, -0.001, -0.967, 0.045)
position_approche_bleu = JointsPosition(-2.565, -0.773, 0.065, -0.002, -0.860, -1.568)
Position_intermediaire_prise_depose_plaque = JointsPosition(-2.260, -0.881, 0.482, -0.001, -1.170, -0.612)
Position_depose_plaque = JointsPosition(-1.639, -0.252, -0.671, -0.005, -0.645, 0.937)

# Première boucle - pick & place de 2 jetons bleus
for count in range(2):
    robot.move(position_approche_bleu)
    robot.move(position_prise_bleu)
    robot.grasp_with_tool()
    robot.move(Position_intermediaire_prise_depose_plaque)
    robot.move(Position_depose_plaque)
    robot.release_with_tool()


# Deuxième boucle - pick & place de 3 jetons verts
for count in range (3):
    robot.move(Position_approche_vert)
    robot.move(Position_prise_vert)
    robot.grasp_with_tool()
    robot.move(Position_intermediaire_prise_depose_plaque)
    robot.move(Position_depose_plaque)
    robot.release_with_tool()
```

Activité 3: Boucles et convoyeur

Préparation:

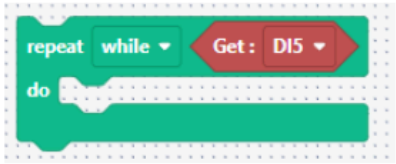
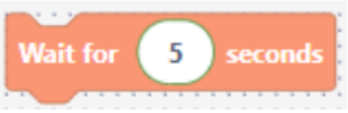
Téléchargez les programmes suivants:

- Blockly 👉 Programme "Conveyor while loop" .json
- Python 👉 Programme "Conveyor while loop" .py

Contenu de l'activité:

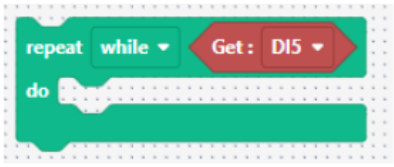
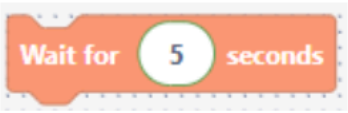
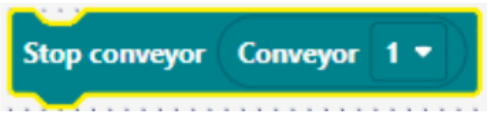
Comme précédemment, vous pouvez vous familiariser avec le programme Python.

A vous de compléter le tableau d'équivalence suivant:

Blockly	Python
	
	
	
	

Explication

Tableau d'équivalences

Blockly	Python
	`robot.run_conveyor(conveyor_id, speed=100, direction=ConveyorDirection.BACKWARD)`
	`while robot.digital_read(sensor_pin_id) == False:`
	`robot.wait(5)`
	`robot.stop_conveyor(conveyor_id)`

💡 Plus en détails:

Contrôle du convoyeur:

```
robot.run_conveyor(conveyor_id, speed=100, direction=ConveyorDirection.BACKWARD)
```

`run_conveyor()`

C'est une **méthode (fonction attachée à l'objet robot)** qui sert à **faire tourner le convoyeur**.

Arguments de la méthode

1. `conveyor_id`
   - L'identifiant du convoyeur que vous voulez contrôler.
   - Si votre robot a plusieurs convoyeurs, chaque convoyeur a son ID.
2. `speed=100`
   - La vitesse à laquelle le convoyeur doit tourner.
   - Ici `100` signifie **vitesse maximale** (les valeurs peuvent aller typiquement de 0 à 100).
3. `direction=ConveyorDirection.BACKWARD`
   - La direction de rotation du convoyeur.
   - `ConveyorDirection.BACKWARD` signifie **marche arrière**.
   - L'autre option possible est `ConveyorDirection.FORWARD` pour aller **dans le sens normal**.

Résumé en langage simple :

> « Le robot fait tourner le convoyeur identifié par `conveyor_id` à **100% de vitesse**, **en marche arrière**. »

Boucle while avec IR Sensor:

```
while robot.digital_read(sensor_pin_id) == False:
```

`while`

- C'est une **boucle** en Python.
- Elle répète **le bloc de code qui suit** tant que la condition est **vraie**.
- Ici, tout ce qui est indenté sous le `while` sera exécuté **en boucle**.

`robot.digital_read(sensor_pin_id)`

- C'est une **fonction qui lit l'état d'une entrée numérique** (Digital Input) du robot.
- `sensor_pin_id` correspond à **l'ID du capteur** branché sur le robot (ici: le capteur infrarouge identifié sous le DI n°5).
- La fonction renvoie **True** ou **False** selon si le capteur détecte quelque chose ou non.

Par défaut, le capteur infrarouge renvoie la donnée "True". Donc s'il détecte quelque chose, il passera en "False"

`== False`

- On vérifie si la lecture du capteur est **False**.
- Pour notre capteur:
  - `True` → rien devant le capteur
  - `False` → un objet est détecté

Donc `while ... == False` signifie : **tant qu'un objet n'est pas détecté**.

Résumé en langage simple :

> « Tant que le capteur identifié par `sensor_pin_id` **ne détecte rien**, répétez le bloc de code qui suit. »

Wait

```
robot.wait(5)
```

`wait()`

- C'est une **méthode (fonction attachée à l'objet robot)** qui sert à **faire une pause dans l'exécution du programme**.
- Le robot **ne fait rien** pendant cette pause.

`(5)`

- C'est le **paramètre de la fonction**, ici la durée de la pause.
- `5` signifie **5 secondes**.
- Vous pouvez mettre n'importe quel nombre positif pour définir la durée souhaitée, toujours en secondes

Résumé en langage simple :

> « Le robot **attend 5 secondes** avant de passer à la commande suivante. »

💡 **Astuce pratique :**

Cette fonction est très utile pour :

- Laisser le temps à un convoyeur de se déplacer.
- Attendre qu'un objet soit bien positionné avant de continuer.
- Synchroniser plusieurs actions du robot.

Arrêt du convoyeur

```
robot.stop_conveyor(conveyor_id)
```

`stop_conveyor()`

- C'est une **méthode (fonction attachée à l'objet robot)** qui sert à **arrêter le convoyeur**.
- Elle stoppe immédiatement le mouvement du convoyeur.

`(conveyor_id)`

- C'est le **paramètre de la fonction**, ici l'identifiant du convoyeur que vous voulez arrêter.
- Si votre robot a plusieurs convoyeurs, chaque convoyeur a son **ID**.

Résumé en langage simple :

> « Le robot **arrête le convoyeur identifié par** `conveyor_id` immédiatement. »

💡 **Astuce pratique :**

- Utilisez cette commande après une boucle de détection avec un capteur IR pour stopper le convoyeur dès qu'un objet est à la bonne position.

Activité 4: Conditions et fonctions

Préparation:

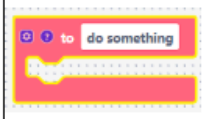
Téléchargez les programmes suivants:

- Blockly 👉 Programme "Condition & Fonction" .json
- Python 👉 Programme "Conditions & Fonctions" .py

Contenu de l'activité:

Comme précédemment, vous pouvez vous familiariser avec le programme Python.

1/ A vous de compléter le tableau d'équivalence suivant:

Blockly	Python
	
	
	

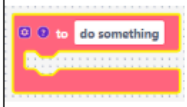
2/ Modifications du programme Python

Dans le programme Python, apporter les modifications suivantes pour, en ayant manuellement placé 3 jetons sur la plaque vision, de chaque couleur, saisir avec la vision 1 jeton rouge

Explication

1/ Tableau d'équivalences

Blockly	Python

	`def nom_fonction(param1, param2, param3)`
	`  param1 = ...`
	`  param2 = ...`
	`  param3 = ...`
	
	`Appel de la fonction:`
	
	`nom_fonction(param 1, param2, param 3)`

	`if`
	`obj_found, shape_ret, color_ret = robot.vision_pick('bundle_workspace')`

💡 **Plus de détails:**

Le `if` : test de condition

```
if obj_found and color_ret == ObjectColor.BLUE
```

En texte:

> “**Si** un objet a été trouvé **et** que sa couleur est bleue, alors fais l’action suivante.”

En particulier:

- `if` = mot-clé de Python pour **tester une condition**
- `obj_found` est une **variable booléenne,** c'est à dire une valeur qui ne peut renvoyer que deux valeurs soit `True` ou `False`, renvoyée par `robot.vision_pick()`
  - `True` → le robot a trouvé un objet
  - `False` → rien détecté
- `and` signifie “et”
- `color_ret == ObjectColor.BLUE` compare la couleur détectée (`color_ret`) à la constante qui représente le **bleu**

➡️ Donc la condition est vraie **uniquement** si :

- un objet a bien été détecté **et**
- cet objet est **bleu**.

L’action si la condition est vraie:

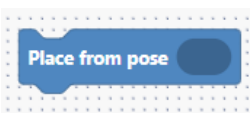
```
robot.place(Position_depose_convoyeur)
```

Cette ligne est **exécutée uniquement si le `if` est vrai**.

Dans `pyniryo`, la méthode `place()` ordonne au robot :

1. d’aller à la position donnée (`Position_depose_convoyeur`)
2. et d’y **déposer** l’objet qu’il tient

Cela combine un mouvement + un relâchement du gripper (ou de la pince).

C'est l'équivalent dans Blockly de: 

En résumé

Élément	Signification
`if`	test logique
`obj_found`	résultat de la vision (`True`/`False`)
`and`	logique “ET”
`color_ret == ObjectColor.BLUE`	vérifie si la couleur détectée est bleue
`robot.place(...)`	dépose l’objet à la position donnée
`# ...`	commentaire explicatif, non exécuté

2/ Modifications du programme Python

```
# !/usr/bin/env python

from pyniryo import NiryoRobot, JointsPosition, PoseObject, ObjectColor

robot = NiryoRobot('127.0.0.1')

robot.calibrate_auto()

# Variables liées aux positions
Position_depose_convoyeur = PoseObject(0.2092,-0.136,0.167,7.5398,89.9231,-170.3682)

# Variables liées à la vision
workspace_name = "bundle_workspace"
observation_pose = JointsPosition(-1.600, -0.054, 0.127, -0.076, -1.918, 0.149)


# Pick & place vision
# Se déplacer dans une position d'observation puis
robot.move(observation_pose)

# Prise du jeton avec la vision:

obj_found, shape_ret, color_ret = robot.vision_pick('bundle_workspace', color=ObjectColor.RED)
if obj_found and color_ret==ObjectColor.RED:        # Si le robot réussi à saisir un objet de couleur BLEUE
alors déposer sur le convoyeur
    robot.place(Position_depose_convoyeur)
```

Activité 2 – Prédire le comportement du robot

Objectif :

Lire et comprendre un programme Python pour anticiper les mouvements du robot.

Challenge #1

Prenez connaissance du programme Python suivant:

Nous avons supprimé les commentaires du codes exprès afin de rendre l'exercice plus complexe. Cela montre bien aussi l'utilité des commentaires quand on partage son script avec d'autres.

```
# !/usr/bin/env python

from pyniryo import NiryoRobot, JointsPosition, PoseObject, ObjectColor

robot = NiryoRobot('127.0.0.1')

robot.calibrate_auto()

position_prise_bleu = JointsPosition(-2.542, -0.948, 0.101, -0.002, -0.720, -1.651)
Position_approche_vert = JointsPosition(-2.343, -1.047, 0.442, -0.001, -0.967, 0.045)
Position_prise_vert =JointsPosition(-2.357, -1.199, 0.593, -0.001, -0.963, 0.709)
position_approche_bleu = JointsPosition(-2.565, -0.772, 0.065, -0.002, -0.860, -1.567)
Position_intermediaire_prise_depose_plaque = JointsPosition(-2.260, -0.881, 0.482, -0.001, -1.170, -0.612)
Position_depose_plaque = JointsPosition(-1.639, -0.252, -0.671, -0.005, -0.645, 0.937)
Position_depose_convoyeur = PoseObject(0.2092,-0.136,0.167,7.5398,89.9231,-170.3682)
workspace_name = "bundle_workspace"
observation_pose = JointsPosition(-1.600, -0.054, 0.127, -0.076, -1.918, 0.149)

for count in range (2):
    robot.move(Position_approche_vert)
    robot.move(Position_prise_vert)
    robot.grasp_with_tool()
    robot.move(Position_intermediaire_prise_depose_plaque)
    robot.move(Position_depose_plaque)
    robot.release_with_tool()
robot.move(position_approche_bleu)
robot.move(position_prise_bleu)
robot.grasp_with_tool()
robot.move(Position_intermediaire_prise_depose_plaque)
robot.move(Position_depose_convoyeur)
robot.release_with_tool()
```

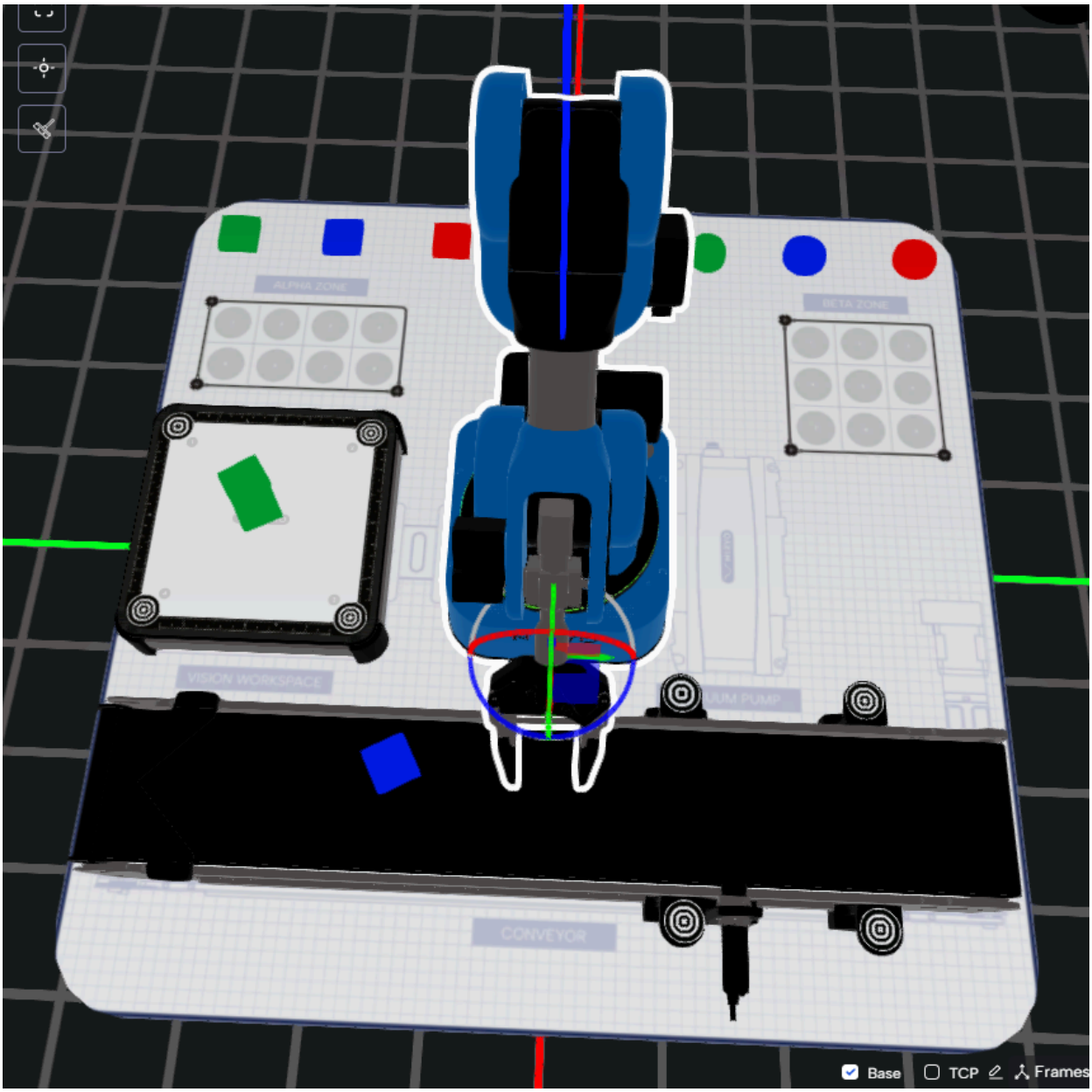
Challenge:

Sans exécuter le code sur NiryoStudio, prédisez ce que fera le robot à chaque étape.

Solution:

Comparez vos prédictions avec l'exécution réelle du robot.

Le robot prend et déplace 2 carrés vert sur la plaque vision et 1 carré bleu sur le convoyeur.


ALPHA ZONE
BETA ZONE
VISION WORKSPACE
CONVEYOR
Base
TCP
Frames

Challenge #2

Prenez connaissance du programme Python suivant:

Nous avons supprimé les commentaires du codes exprès afin de rendre l'exercice plus complexe. Cela montre bien aussi l'utilité des commentaires quand on partage son script avec d'autres.

```
# !/usr/bin/env python3

from pyniryo import NiryoRobot, JointsPosition, PoseObject, ObjectColor

robot = NiryoRobot('127.0.0.1')

robot.calibrate_auto()

position_prise_bleu = JointsPosition(-2.542, -0.948, 0.101, -0.002, -0.720, -1.651)
Position_approche_vert = JointsPosition(-2.343, -1.047, 0.442, -0.001, -0.967, 0.045)
Position_prise_vert =JointsPosition(-2.357, -1.199, 0.593, -0.001, -0.963, 0.709)
position_approche_bleu = JointsPosition(-2.565, -0.772, 0.065, -0.002, -0.860, -1.567)
Position_intermediaire_prise_depose_plaque = JointsPosition(-2.260, -0.881, 0.482, -0.001, -1.170, -0.612)
Position_depose_plaque = JointsPosition(-1.639, -0.252, -0.671, -0.005, -0.645, 0.937)
Position_depose_convoyeur = PoseObject(0.2092,-0.136,0.167,7.5398,89.9231,-170.3682)
workspace_name = "bundle_workspace"
observation_pose = JointsPosition(-1.600, -0.054, 0.127, -0.076, -1.918, 0.149)


robot.move(position_approche_bleu)
robot.move(position_prise_bleu)
robot.grasp_with_tool()
robot.move(Position_intermediaire_prise_depose_plaque)
robot.move(Position_depose_plaque)
robot.release_with_tool()
robot.move(observation_pose)
obj_found, shape_ret, color_ret = robot.vision_pick('bundle_workspace', color=ObjectColor.GREEN)
if obj_found and color_ret==ObjectColor.GREEN:
    robot.place(Position_depose_convoyeur)
```

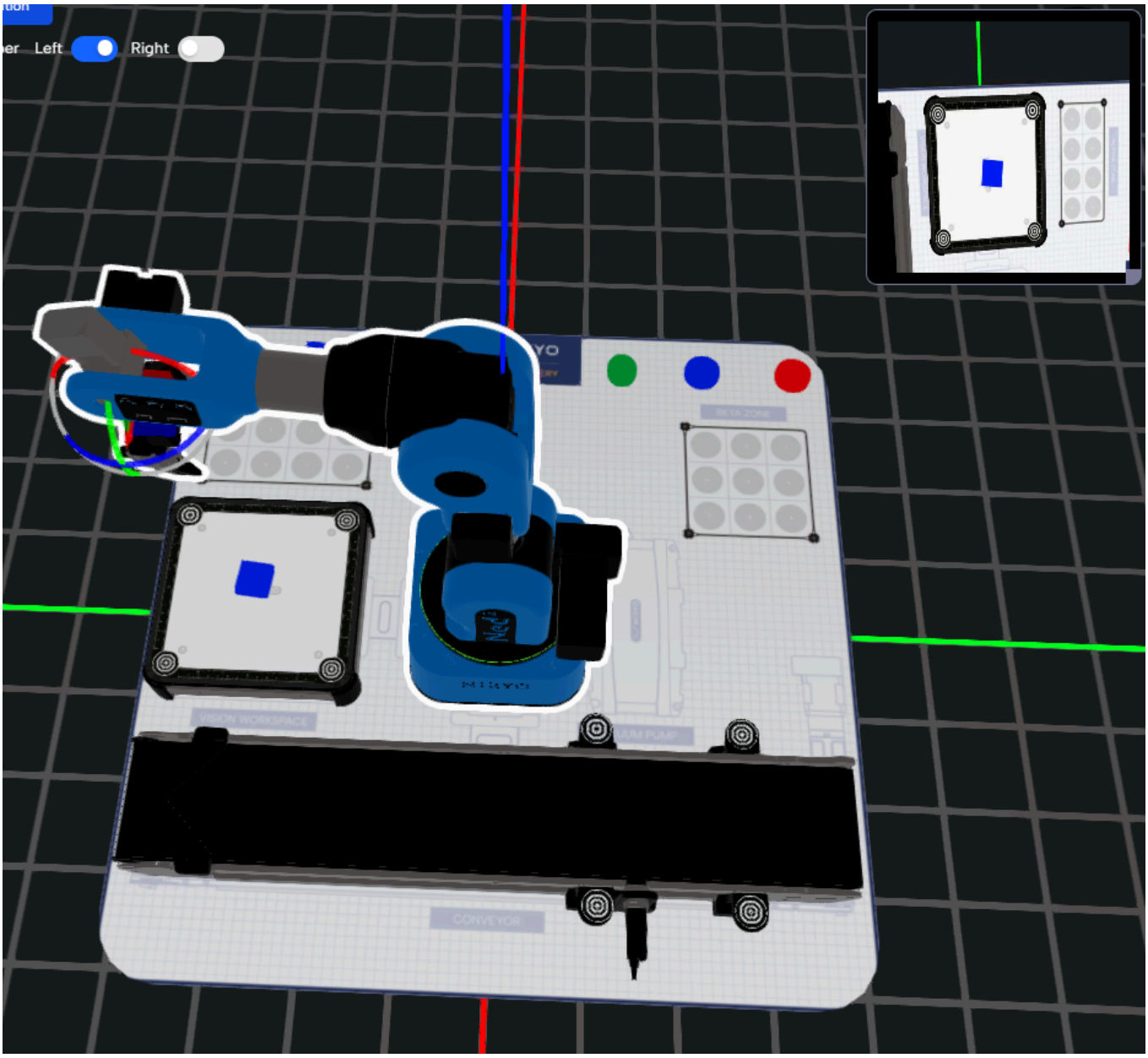
Challenge:

Sans exécuter le code sur NiryoStudio, prédisez ce que fera le robot à chaque étape.

Solution:

Comparez vos prédictions avec l'exécution réelle du robot.

Le robot prend et déplace 1 carré bleu sur la plaque vision. Il essaie ensuite de saisir un jeton vert avec le système de vision. Comme il n'observe qu'un jeton bleu, il n'exécute pas la commande et le programme prend fin.


Left
Right

Challenge #3

Prenez connaissance du programme Python suivant:

Nous avons supprimé les commentaires du codes exprès afin de rendre l'exercice plus complexe. Cela montre bien aussi l'utilité des commentaires quand on partage son script avec d'autres.

```
# !/usr/bin/env python3

from pyniryo import NiryoRobot, JointsPosition, PoseObject, ObjectColor

robot = NiryoRobot('127.0.0.1')

robot.calibrate_auto()

position_prise_bleu = JointsPosition(-2.542, -0.948, 0.101, -0.002, -0.720, -1.651)
Position_approche_vert = JointsPosition(-2.343, -1.047, 0.442, -0.001, -0.967, 0.045)
Position_prise_vert =JointsPosition(-2.357, -1.199, 0.593, -0.001, -0.963, 0.709)
position_approche_bleu = JointsPosition(-2.565, -0.772, 0.065, -0.002, -0.860, -1.567)
Position_intermediaire_prise_depose_plaque = JointsPosition(-2.260, -0.881, 0.482, -0.001, -1.170, -0.612)
Position_depose_plaque = JointsPosition(-1.639, -0.252, -0.671, -0.005, -0.645, 0.937)
Position_depose_convoyeur = PoseObject(0.2092,-0.136,0.167,7.5398,89.9231,-170.3682)
workspace_name = "bundle_workspace"
observation_pose = JointsPosition(-1.600, -0.054, 0.127, -0.076, -1.918, 0.149)


def sequence(approche_pick, pick, approche_depose, depose):
    robot.move(approche_pick)
    robot.move(pick)
    robot.grasp_with_tool()
    robot.move(approche_depose)
    robot.move(depose)
    robot.release_with_tool()
sequence(Position_approche_vert,Position_prise_vert,Position_intermediaire_prise_depose_plaque,
Position_depose_plaque)
robot.move(observation_pose)
obj_found, shape_ret, color_ret = robot.vision_pick('bundle_workspace')
if obj_found and color_ret==ObjectColor.GREEN:
    robot.place(Position_depose_convoyeur)
```

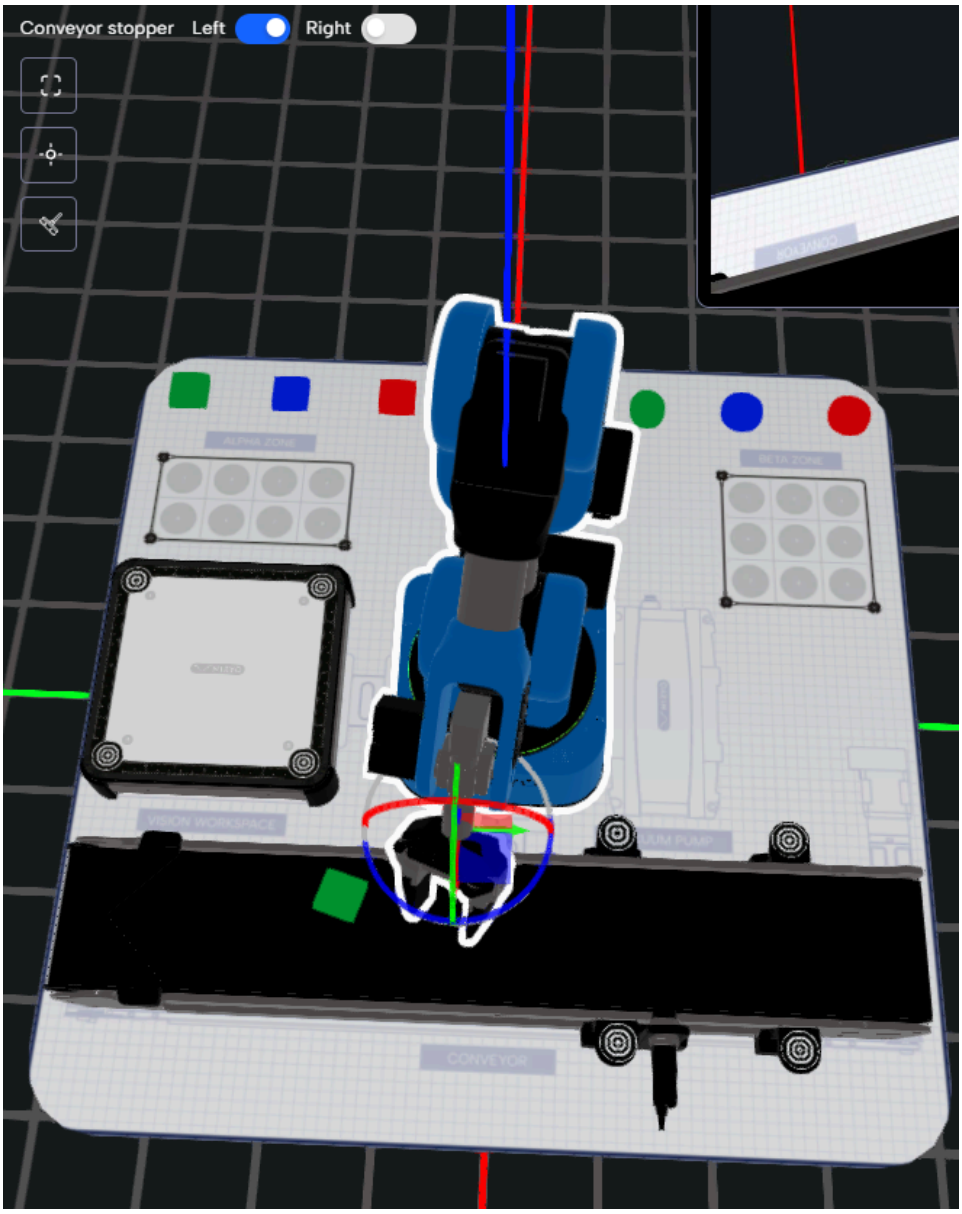
Challenge:

Sans exécuter le code sur NiryoStudio, prédisez ce que fera le robot à chaque étape.

Solution:

Comparez vos prédictions avec l'exécution réelle du robot.

Le robot prend et déplace 1 carré vert sur la plaque vision à l'aide d'une fonction appelée "sequence". A l'aide de la vision, il saisit un jeton vert sur la plaque et le déplace sur le convoyeur.


Conveyor stopper
Left
Right
ALPHA ZONE
BETA ZONE
VISION WORKSPACE
CONVEYOR

Challenge #4

Prenez connaissance du programme Python suivant:

Nous avons supprimé les commentaires du codes exprès afin de rendre l'exercice plus complexe. Cela montre bien aussi l'utilité des commentaires quand on partage son script avec d'autres.

```
# !/usr/bin/env python3

from pyniryo import NiryoRobot, JointsPosition, PoseObject, ObjectColor

robot = NiryoRobot('127.0.0.1')

robot.calibrate_auto()

position_prise_bleu = JointsPosition(-2.542, -0.948, 0.101, -0.002, -0.720, -1.651)
Position_approche_vert = JointsPosition(-2.343, -1.047, 0.442, -0.001, -0.967, 0.045)
Position_prise_vert =JointsPosition(-2.357, -1.199, 0.593, -0.001, -0.963, 0.709)
position_approche_bleu = JointsPosition(-2.565, -0.772, 0.065, -0.002, -0.860, -1.567)
Position_intermediaire_prise_depose_plaque = JointsPosition(-2.260, -0.881, 0.482, -0.001, -1.170, -0.612)
Position_depose_plaque = JointsPosition(-1.639, -0.252, -0.671, -0.005, -0.645, 0.937)
Position_depose_convoyeur = PoseObject(0.2092,-0.136,0.167,7.5398,89.9231,-170.3682)
Position_prise_rouge = JointsPosition(-2.7985,-0.7958,-0.1992,0,-0.5753,0.2071)
Position_approche_rouge = JointsPosition(-2.8046,-0.6186,-0.1871,0,-0.764,0.201)
workspace_name = "bundle_workspace"
observation_pose = JointsPosition(-1.600, -0.054, 0.127, -0.076, -1.918, 0.149)


def sequence(approche_pick, pick, approche_depose, depose):
    robot.move(approche_pick)
    robot.move(pick)
    robot.grasp_with_tool()
    robot.move(approche_depose)
    robot.move(depose)
    robot.release_with_tool()
for count in range(2):
    sequence(Position_approche_rouge, Position_prise_rouge, Position_intermediaire_prise_depose_plaque,
Position_depose_plaque)
for count in range(2):
    sequence(Position_approche_vert, Position_prise_vert, Position_intermediaire_prise_depose_plaque,
Position_depose_convoyeur)
robot.move(observation_pose)
obj_found, shape_ret, color_ret = robot.vision_pick('bundle_workspace')
if obj_found and color_ret==ObjectColor.RED:
    robot.place(Position_depose_convoyeur)
```

Challenge:

Sans exécuter le code sur NiryoStudio, prédisez ce que fera le robot à chaque étape.

Solution:

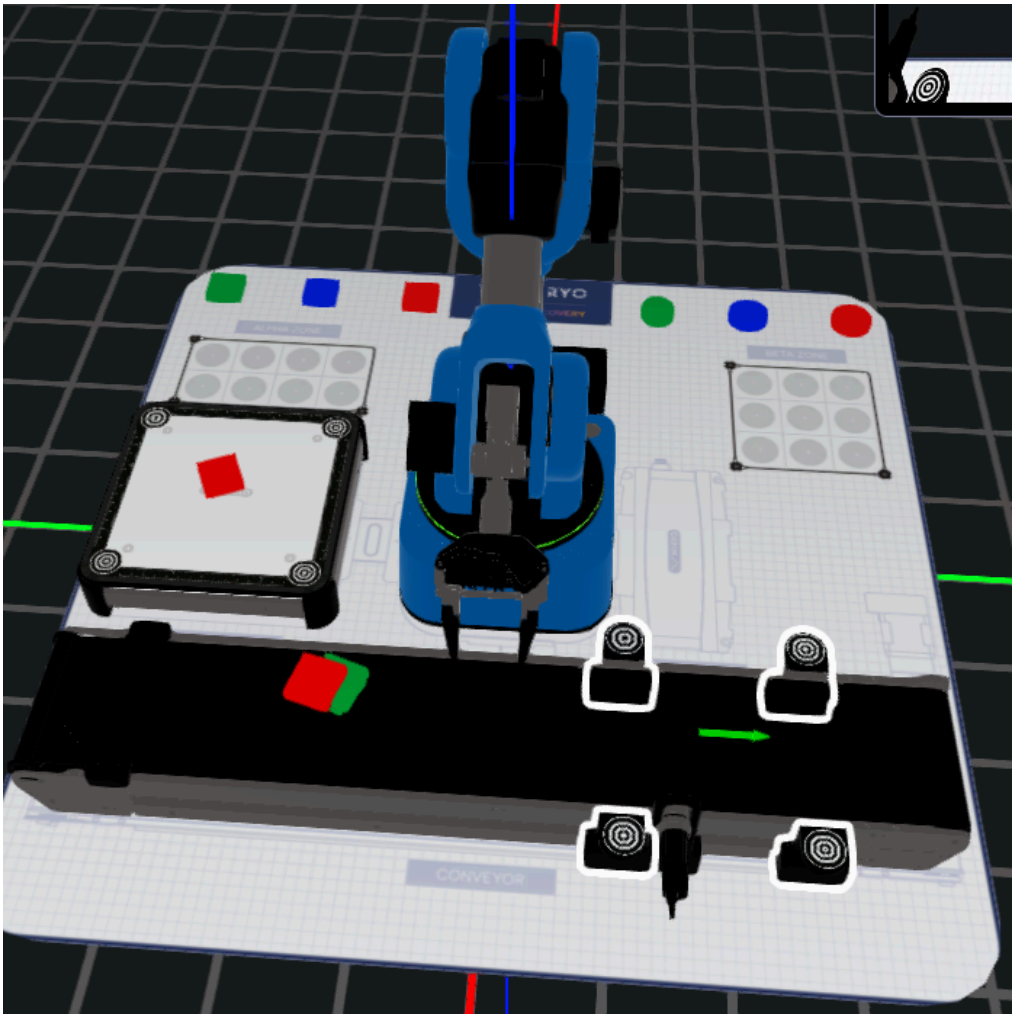
Comparez vos prédictions avec l'exécution réelle du robot.

Le robot prend et déplace 2 carrés rouges sur la plaque vision à l'aide d'une fonction appelée "sequence" exécutée dans une boucle qui répète la fonction 2x.

Ensuite il saisit et déplace 2 carrés verts sur le convoyeur, de la même manière.

A l'aide de la vision, le robot saisit un jeton rouge sur la plaque et le déplace sur le convoyeur.

Au total, j'aurais 1 carré rouge sur la plaque vision et 2 carrés verts et 1 carré rouge sur le convoyeur.



Activités supplémentaires

Nous vous proposons de concevoir d'autres activités pour initier les étudiants au langage Python.

En fonction de l'organisation de votre classe, du niveau de vos groupes, ces activités peuvent être adaptées:

Activité 3 – Traduction manuelle Blockly → Python

Objectif : Comprendre la logique de la conversion Blockly → Python en la réalisant eux-mêmes.

Déroulé :

1. Le professeur donne un programme Blockly simple (ex. pick & place).
2. **Challenge :** Traduire le programme en Python ligne par ligne.
3. Les étudiants peuvent travailler seuls ou en binôme.
4. Vérification : exécuter le code Python généré et comparer le résultat avec le programme Blockly original.
5. Discussion sur les différences de syntaxe, indentation, et noms de variables.

Activité 4 – Challenge collaboratif Blockly ↔ Python

Objectif : Renforcer la compréhension et la communication autour de la traduction Blockly ↔ Python.

Déroulé :

1. Former des binômes : chaque binôme aura deux rôles :
   - Étudiant A écrit un programme Blockly.
   - Étudiant B traduit ce programme en Python.
2. Puis inversion des rôles : Étudiant B écrit un nouveau programme Blockly, A le traduit en Python.
3. **Challenge bonus :** Ajouter une petite modification dans Blockly pour tester la réactivité du partenaire et la compréhension des modifications dans Python.
4. Discussion finale : les étudiants présentent leur travail et expliquent comment ils ont traduit la logique des blocs en code Python.